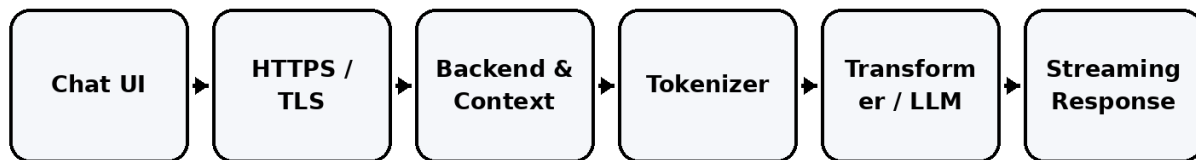


What Happens Technically When You Type in a Chat Window?

A technical introduction to LLM request processing, Transformers, token generation, tools and RAG

What Happens After You Press Enter



1. The End-to-End Pipeline

When you type a prompt and press Enter, the application does much more than send a sentence directly to a language model. A simplified flow is: your text → chat frontend → encrypted network request → backend/orchestration → context assembly → tokenization → model inference → next-token generation → streaming → displayed response.

The exact production architecture is implementation-dependent, but this model is a useful technical representation of how modern LLM chat systems operate.

2. The Chat Client Sends Your Message

Your message initially exists as Unicode text in the browser or mobile/desktop application. When submitted, the client sends a request to a backend service, normally over HTTPS protected by TLS. The request identifies the conversation and carries the new message plus application-level metadata needed to process it.

Conceptually, the backend may receive data similar to:

Current user message: "Explain cloud computing in simple terms."

3. Context Assembly

The model normally receives more than the latest sentence. The system assembles an input context containing relevant system/developer instructions, conversation history, the current user message, and—when appropriate—results from tools or retrieved documents.

System instructions

Previous user message

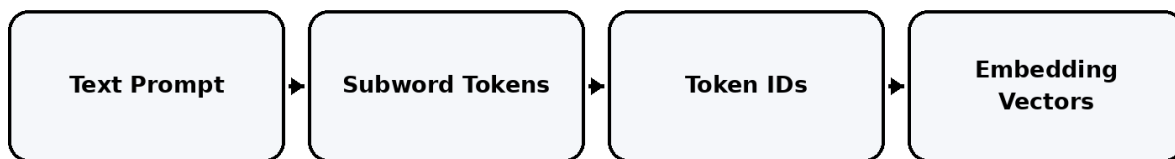
Previous assistant response

Current user message

Optional tool/retrieval context

4. Tokenization: Text Becomes Numbers

Tokenization Pipeline



An LLM does not directly calculate on words. A tokenizer splits the text into tokens, often words, parts of words, punctuation, or other frequently occurring byte/subword sequences. Each token maps to an integer token ID. Exact token boundaries and IDs depend on the tokenizer used by the model.

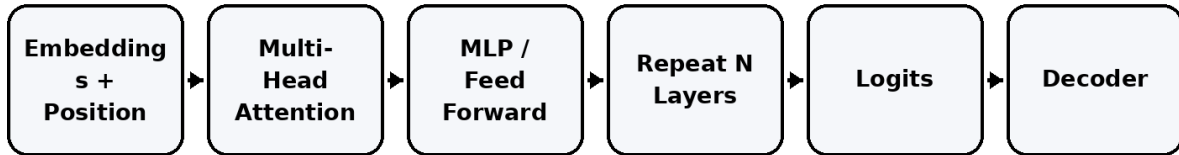
Text → ["Explain", " cloud", " computing", ...] → token IDs → vectors

5. Embeddings: Token IDs Become Vectors

Token IDs are indices, not semantic quantities. The model uses an embedding table to map each token ID to a high-dimensional vector. These vectors are the numerical representations processed by the neural network. Positional information is also incorporated so the model can distinguish token order.

6. Transformer Processing

Simplified Transformer Inference

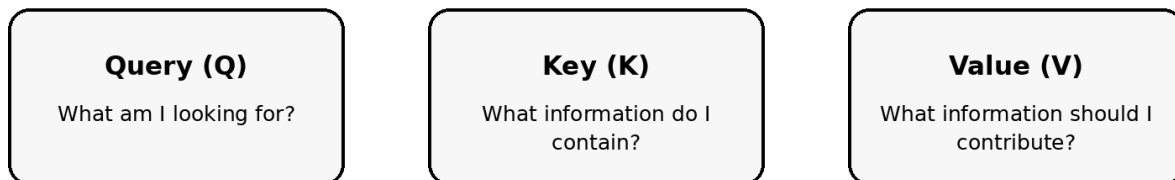


The sequence of vectors passes through many Transformer layers. Each layer typically combines attention-related operations with feed-forward/MLP computation, normalization, and residual connections. The repeated layers progressively transform the representation of every token based on its context.

7. Self-Attention and Q/K/V

Self-Attention: Q, K and V

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$



Attention weights determine how strongly each token uses information from earlier context.

Self-attention allows a token representation to incorporate information from other relevant positions. The model derives Query (Q), Key (K), and Value (V) vectors through learned linear projections. A standard scaled dot-product attention expression is:

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

In a causal language model, masking prevents a position from attending to future tokens during generation. Multi-head attention performs several attention calculations in parallel, allowing different heads to capture different relationships.

8. From Hidden State to Next-Token Probabilities

After the final Transformer layer, the model produces scores called logits over its vocabulary. A softmax operation can convert logits into a probability distribution. A decoding strategy then selects a token. Depending on configuration, decoding may be deterministic or may sample from likely tokens using controls such as temperature or top-p.

Possible next token	Illustrative probability
access	0.31
use	0.18
store	0.12
run	0.09
purchase	0.01

These probabilities are illustrative only; they are not actual outputs from a specific model run.

9. Autoregressive Generation

Generation is autoregressive: the model selects one token, appends it to the sequence, and predicts the next token again. For example:

Cloud computing allows users to

- access
- computing
- resources
- over
- the Internet.

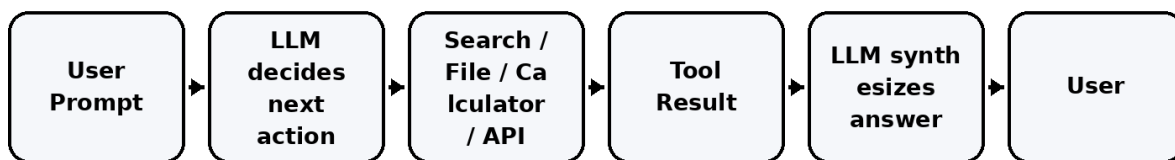
Modern inference engines optimize this repeated process with techniques such as KV caching, batching, and specialized GPU/accelerator kernels.

10. Why the Answer Appears Gradually

The backend can stream generated text to the client as tokens or small chunks become available. The browser/app renders these chunks progressively, which is why a response appears to be typed rather than arriving only after the complete answer has been generated.

11. Tool-Augmented Chat

Tool-Augmented Chat Flow



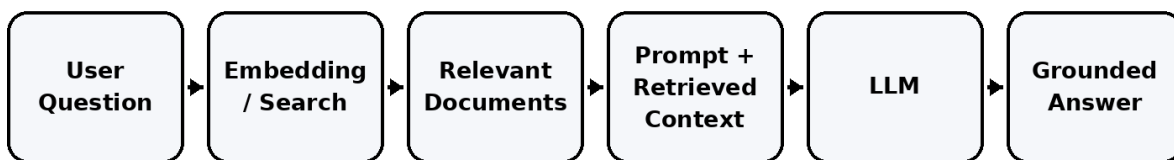
A chat system may allow the model to request tools when the task requires information or computation outside normal model inference. Examples include web search, calculators, file retrieval, databases, code execution, or application APIs. The tool result is returned to the orchestration layer and supplied back to the model as additional context before it produces the user-facing answer.

12. Where the Model's Learned Knowledge Resides

A trained LLM is not simply a database of stored answers. During training, optimization adjusts a very large set of numerical parameters (weights). During inference, these learned weights participate in matrix operations that transform the input representations and produce next-token scores. The model can still be combined with external data systems when current or private information is required.

13. RAG: Retrieval-Augmented Generation

RAG: Retrieval-Augmented Generation



Retrieval-Augmented Generation (RAG) supplements the LLM with external information at inference time. A retrieval system searches a document collection for passages relevant to the user's question. Those passages are inserted into the model's context, allowing the model to answer using information that does not have to be encoded in its trained parameters.

A common implementation uses document chunking, embeddings, a vector or hybrid search index, retrieval, prompt construction, and generation. RAG does not inherently guarantee correctness; retrieval quality and grounding instructions still matter.

14. Complete Technical Summary

1. User enters Unicode text in the chat interface.
2. The client sends the request through an encrypted connection.
3. The backend assembles instructions, conversation context, and optional external context.
4. The tokenizer converts text into token IDs.

5. Embeddings and positional information create vector representations.
6. Transformer layers process the sequence using attention and MLP operations.
7. The final representation is projected to vocabulary logits.
8. A decoding strategy selects the next token.
9. The selected token is appended and generation repeats autoregressively.
10. Output is streamed back to the client and rendered in the chat window.
11. When needed, tool calls or RAG retrieval can add external information to the inference loop.

15. Key Terms

Term	Meaning
Token	A unit of text processed by the model.
Tokenizer	Software that maps text to token IDs and back.
Embedding	A learned numerical vector representation.
Transformer	The neural-network architecture underlying most modern LLMs.
Attention	A mechanism for combining information from relevant token positions.
Logit	An unnormalized score for a possible output token.
Softmax	A function that converts scores into a probability distribution.
Inference	Running a trained model to produce outputs.
RAG	Retrieval-Augmented Generation: retrieval of external context before generation.
Tool calling	A mechanism that lets a model request an external function/service.

16. One-Line Mental Model

**Prompt → Context → Tokens → Embeddings → Transformer → Logits → Decoding → Next
Token → Repeat → Stream**